

EXPLICACIÓ LAB2 - DAVID PÉREZ CRUZ

CODI TT.C:

En aquesta part del codi el que fem és simplement assignar els valors corresponents a "TEAMNAME I ENIGMA1" i comprovar que el número de caràcters sigui igual a 3 (sino mostra un error).

Validem que els números passats al programa siguin enters majors a 0, utilitzant la funció "nombre_valid" y si son correctes, els hi assignem els valor de M y de N.

```
#define TEAMNAME "davidperezcruz"
#define ENIGMA1 "protomolecule"

// Pre: nombre a comprovar si és enter
// Post: retorna true si és enter més gran que 0, false altrament
int nombre_valid(char *str) {
    int num = atoi(str);
    return num > 0;
}

int main(int argc, char *argv[]) {

    if (argc != 3) { // Comprovem que únicament es passin 3 paràmetres
        printf("Ús: ./tt M > 0 N > 0 \n");
        exit(-1);
    }

    if (!nombre_valid(argv[1]) || !nombre_valid(argv[2])) { //
Verificació d'enters positius
        printf("Error: M i N han de ser enters més grans que 0\n");
        exit(-1);
    }

    int M = atoi(argv[1]);
    int N = atoi(argv[2]);
```

Tot seguit, el que fem es obrir el banc de temps utilitzant `dummy_open` i fem la comprovació per si s'ha obert correctament.

Creem els replicants, creem un array per guardar els codis de sortida per després mostrar-los.

Fem un bucle per a tots els replicants, on a `pid` fem la clonació utilitzant `fork()`. Si el `pid` es igual a 0, significa que es un fill y per tant, fem un `close` per tancar l'entrada estàndard, i un `dup(fd)` per duplicar el file descriptor per el replicant, després executem `./replicant`, si dona error sortim amb un `exit(-1)`; Si el `pid` es major a 0, significa que es el pare, llavors continua un altre cop amb el bucle.

Si el `pid` no es ni igual a 0, ni major, significa que ha donat un error en el `fork` i fem `exit(-1)`.

```
int fd = dummy_open(); // Abrir el banco de tiempo
if (fd < 0) {
    perror("Error obrint el banc de temps");
    exit(-1);
}

// Crear tots els replicants
for (int i = 0; i < M; i++) {
    int pid = fork(); // Crear un replicant

    if (pid == 0) { // Código para el hijo replicant
        close(0); // Cerrar la entrada estándar
        dup(fd); // Duplicar el file descriptor para el
replicant
        execlp("./replicant", "./replicant", argv[2], (char*)0); //
Ejecutar el replicant
        perror("Error exec replicant"); // Si execlp falla
        exit(-1);
    } else if (pid > 0) { // El padre continua
        // Continúa el ciclo para crear más replicants
    } else { // Si fork falla
        perror("Error fork");
        exit(-1);
    }
}
```

Per finalitzar, creem una variable per contenir totes les acumulacions dels valors dels èxits.

Acte seguit, el que fem és un altre bucle per a esperar que cada replicant acabi amb “wait” i sumem el codi de sortida a la variable “acum”.

Una vegada acabat el bucle, fem la comprovació amb dummy_test i fem un dummy_exit.

```
int acum = 0;
    // Esperar a que tots els replicants acabin i acumular els codis de
sortida
    for (int i = 0; i < M; i++) {
        int stat;
        int pid = wait(&stat); // Espera que cada replicant acabi
        if (WIFEXITED(stat)) {
            int exit_code = WEXITSTATUS(stat); // Obtenim el codi de
sortida
            acum += exit_code; // Acumulem els codis de sortida
            printf("Replicant %d finalitzat amb codi de sortida: %d\n",
pid, exit_code);
        } else {
            printf("Error: replicant %d no ha acabat correctament\n",
pid);
            exit(-1); // Si algun replicant falla, abortem la missió
        }
    }

    // Validación con dummy_test
    dummy_test(acum, ENIGMA1, TEAMNAME);
    dummy_exit();

    printf("Done!\n");
    exit(0);
```

CODI REPLICANT.C:

Al codi del replicant, primer de tot incloem “dummy.h”.

Acte seguit, el que fem és crear un buffer de 200 caràcters i comprovar que N sigui menor a 200, si no ho és imprimeix un error i surt amb un “exit(-1)”;

Si sí que ho és, fem un dummy_init i creem la variable “bytesllegits” per llegir els bites. Fem un bucle, on va llegint el valor i el va posant en “bytesllegits” mentre que aquest valor sigui major a 0.

Llavors fem un dummy_comp per procesar el temps.

Si tot ha sigut exitós, fem un dummy_exit i sortim amb un “exit(0);” com a salida exitosa del replicant.

```
#include "dummy2.h"
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h> // Para usar la función read

int main(int argc, char *argv[]) {
    int N = atoi(argv[1]);
    char buffer[200];

    if (N < 200) {
        dummy_init(buffer, N);
        int bytesllegits;

        // Proceso de robar y procesar tiempo del banco
        while ((bytesllegits = read(0, buffer, N)) > 0) {
            dummy_comp(buffer, bytesllegits); // Procesa el tiempo
            robado
        }

        dummy_exit();
        exit(0); // Salida exitosa del replicante
    } else {
        printf("Error: N excedeix la mida del buffer.\n");
        exit(-1); // Error en la entrada
    }
}
```

COMPROVACIÓ DEL CODI:

```
david@david-Modern-14-C13M:~/2no/SIOP/lab2$ ./gendummy S
*** GENDUMMY: Creating dummy file ... 10% 20% 30% 40% 50% 60% 70% 80% 90%
*** GENDUMMY: Done! [1000000000] bytes
david@david-Modern-14-C13M:~/2no/SIOP/lab2$ gcc tt.c -o tt -L. -ldummy2
david@david-Modern-14-C13M:~/2no/SIOP/lab2$ gcc replicant.c -o replicant -L. -ldummy2
david@david-Modern-14-C13M:~/2no/SIOP/lab2$ ./tt 2 3
*** DUMMY: Opening MASTER dummy file ... [OK]
*** DUMMY: Init CLIENT dummy code...
*** DUMMY: Init CLIENT dummy code...
*** DUMMY: Exit CLIENT dummy code...
*** DUMMY: Exit CLIENT dummy code...
Replicant 8007 finalitzat amb codi de sortida: 100
Replicant 8008 finalitzat amb codi de sortida: 52

*** DUMMY: Testing program execution ... [SUCCEED]
    Good job!

BRONCEKEY: 7a712 [Keep save this key]

*** DUMMY: Exit CLIENT dummy code...
```